

Finding the Right Page

An explanatory packet for `step_7/lib/retrieval.py`

Solanus Casey Archival Tool — `step_7`

Abstract

This packet walks through the retrieval module — the part of the archival tool that, given a question about Father Solanus Casey’s letters and notebooks, returns the handful of passages that answer it, each carrying enough provenance to cite the *exact scanned region*. We build the idea up from first principles: why lexical and semantic search need each other, how Reciprocal Rank Fusion marries them, what a reranker adds, how query transforms (HyDE, RAG-Fusion) and an adaptive router decide how hard to work, and how a Self-RAG/CRAG grader lets an *archive* honestly say “I could not find this.” We close with the three frontier techniques the corpus most wants next — Contextual Retrieval, multimodal/ColPali, and RAPTOR — marking carefully what is wired today versus planned. Wording follows the source comments closely, reorganised into prose.

Contents

1	TL;DR	2
2	Why retrieval is the hard part (and a tiny jargon primer)	2
3	The data shape that makes citation possible	3
4	The pipeline at a glance	3
5	The router: spend brainpower in proportion to the question	4
6	Hybrid retrieval: BM25 and dense, because each rescues the other	4
6.1	BM25 — the lexical workhorse	4
6.2	Dense — the semantic workhorse	5
6.3	Why both, on our corpus	5
7	RRF: fusing ranked lists without calibrating their scores	5
8	Rerank: the expensive judge that reads query and passage together	6
9	Query transforms: HyDE and RAG-Fusion	6
9.1	HyDE — embed a hypothetical <i>answer</i>	6
9.2	RAG-Fusion — many phrasings, one fused result	6
10	Self-RAG / CRAG: earning the right to answer	6
10.1	The free grader (<code>grade_context</code>)	7
10.2	The gated per-document grader (<code>grade_document</code>)	7

11 Putting it together: <code>route_and_retrieve</code>	7
12 The frontier: three techniques this corpus wants next	8
12.1 Contextual Retrieval — the biggest win, sitting <i>upstream</i> of this file	8
12.2 Multimodal / ColPali — retrieve over the page <i>images</i>	8
12.3 RAPTOR — a tree of summaries for global questions	9
13 How to run it, and the honest caveats	9
14 Sources	10

1 TL;DR

The thirty-second version

- **The job.** Turn a question into a short, ranked list of passages, each tagged with `doc_id` → `rid` → `vertices` so we can draw a box on the original scan (an IIIF deep-zoom citation). If the right page never makes the shortlist, no prompt can rescue the answer — retrieval is where accuracy is won or lost.
- **The free path already works.** `router` (heuristic) → BM25 + `dense` → RRF → `grade`. No API key, no paid call, no model download. BM25 nails exact names and dates; dense embeddings catch paraphrase; RRF fuses them without calibrating their incompatible scores.
- **Every upgrade is a toggle, and every paid toggle is gated.** Rerank, HyDE, multi-query/RAG-Fusion, the LLM router, and the per-document grader are all off by default and each routes its cost through `lib/costlog.py`. You opt into spend; you never trip into it.
- **Built for an archive, so it can abstain.** A free Self-RAG/CRAG grader scores the retrieved set (rank margin, lexical overlap, OCR confidence) and returns `answer` / `caveat` / `abstain`. A confident wrong answer is the worst outcome in an archive.
- **The biggest pending win is Contextual Retrieval.** Prepending a one-line context to each terse, date-less notebook entry before indexing is what makes “22 — reports cured.” findable at all. It is an *indexing-time* technique (chunk/embed stage), so it sits upstream of this file — and is the recommended next build.

2 Why retrieval is the hard part (and a tiny jargon primer)

A large language model is a brilliant talker with no memory of our archive. Retrieval-Augmented Generation (RAG) fixes that by *looking things up first*: we find the most relevant passages, hand them to the model, and ask it to answer *only* from what it was given. So the quality of the final answer is capped by the quality of retrieval. Everything in `lib/retrieval.py` exists to raise that cap.

Skip if you already know BM25, embeddings, cosine similarity, and cross-encoders

Sparse / lexical (BM25). Treats text as a bag of words and scores overlap of the *actual tokens*. Great at exact strings — a surname like *O'Donnell*, a date like *1933*.

Dense / semantic (embeddings). Maps a passage to a vector so that passages with similar *meaning* sit close together. Great at paraphrase — it knows *favor*, *miracle*, and *answered prayer* are neighbours.

Cosine similarity. The cosine of the angle between two vectors: +1 same direction (very similar), 0 unrelated, -1 opposite. Our vectors are pre-normalised, so cosine is just a dot product.

Cross-encoder (reranker). Instead of embedding the query and passage separately, it reads them *together* and outputs one relevance score. Much sharper, much slower — so we only ever run it on a short candidate list.

Our corpus makes this concrete and a little brutal. The letters are reasonably well-formed prose, but the notebooks are a running register written in shorthand. A real entry looks like:

"22 — reports cured."

No year. No surname. No condition. On its own that chunk is almost impossible to retrieve — and yet it may be the single record that answers “which favors were reported cured in 1933?” Holding that example in mind explains nearly every design choice below.

3 The data shape that makes citation possible

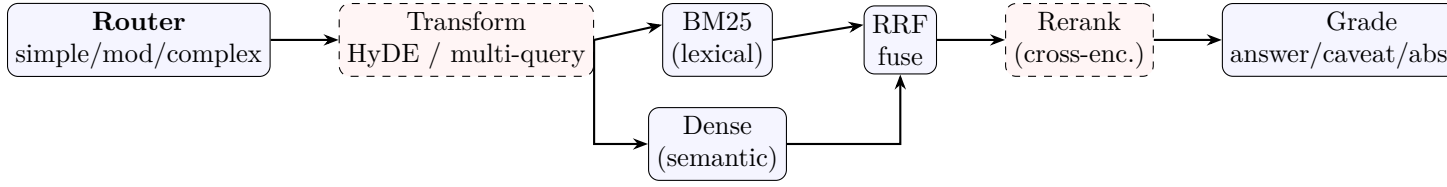
Everything the module returns is a `Hit`: it mirrors the chunk shape from `lib/chunks.py` (`id` / `kind` / `text` / `meta`) and adds a fused `score`, a `provenance` dict, and a `components` debug dict (which retriever ranked it where). The `provenance` is the reason this is an *archival* tool and not a chatbot:

- For a **notebook entry**, the chunk's `meta.rid` points at one region. We look that region up in `_region_index()` — a one-time map of `"doc_id::rid" → {vertices, min_conf, ...}` read from the same `step_6` JSON the chunker reads — and the citation reaches the *exact box* on the page.
- For a **letter** (several regions joined into one chunk) there is no single `rid`, so we record the page-level location and let the UI resolve per-region detail from the source record. The citation still always reaches the right page.

The module is, by design, a set of *pure functions returning ranked Hits*: nothing mutates the corpus, nothing writes files. That keeps each piece testable and lets the dev harness flip any subset on while measuring every model combination.

4 The pipeline at a glance

The adaptive entry point `route_and_retrieve` wires the whole brain together. A query flows left to right; the dashed boxes are paid and *off by default*.



The *free path* is the solid boxes only: **router** → **BM25 + dense** → **RRF** → **grade**. We now take each stage in turn.

5 The router: spend brainpower in proportion to the question

“When was Solanus born?” deserves one cheap lookup. “List every cancer favor reported in 1933 and how each turned out” deserves decomposition, several queries, and synthesis. Treating them identically either wastes money on the easy one or under-serves the hard one.

`route_query` sorts a query into **simple**, **moderate**, or **complex** from cheap signals: length, question words (*who/when/where*), aggregate markers (*every/how many/timeline/compare*), and conjunctions that hint at multi-part asks. It returns a *recommendation*, never a command:

```

{"complexity": "complex",
 "strategy": "decompose -> multi-query retrieval + RRF (+ rerank), then synthesis",
 "reason": "aggregate/multi-part markers or a long, compound question",
 "suggest": {"use_multiquery": True, "multihop": True}}

```

The crucial design point: multi-query is a *paid* LLM call, so the router never spends on its own. The transform runs only if the caller set `use_multiquery`, or if `auto_apply_route=True` (the switch by which an agent opts into letting the router escalate). That is why the default path stays at **exactly zero paid calls even on a “complex” query**. An LLM classifier (`use_llm_router`) handles genuinely ambiguous phrasing and *degrades to the heuristic on any failure*, so the router can never become a hard dependency on the model. *Why it helps us*: the corpus mixes trivial biographical facts with real aggregate archival questions; routing keeps the cheap ones cheap and reserves the heavy machinery for the asks that need it (the literature reports roughly -28% cost and $+8\%$ accuracy from adaptive routing).

6 Hybrid retrieval: BM25 and dense, because each rescues the other

6.1 BM25 — the lexical workhorse

BM25 scores a passage by summing, over each query term, three intuitions multiplied together:

$$\text{score}(d) = \sum_{t \in q} \underbrace{\text{IDF}(t)}_{\text{rare words count more}} \cdot \frac{f_{t,d}(k_1 + 1)}{f_{t,d} + k_1 \left(1 - b + b \frac{|d|}{|D|}\right)},$$

where $f_{t,d}$ is how often term t appears in document d . The numerator rewards term frequency; k_1 *saturates* it so the tenth occurrence matters less than the first; b applies *length normalisation* so a long document does not win merely by being long; and IDF up-weights rare terms. We keep the standard $k_1=1.5$, $b=0.75$.

For us BM25 is unbeatable on *exact tokens*: we tokenise keeping apostrophes (so *O'Donnell* survives) and digits (so *1933* is a real token). We prefer the `rank_bm25` package but ship a small *correct* `BM25Okapi` fallback, so the free path never depends on an install — if the package is missing the module quietly uses the fallback.

6.2 Dense — the semantic workhorse

`dense_search` embeds the query in a chosen *space* (`model@dim`, e.g. `gemini-embedding-001@1536`) and cosine-searches that vector-store partition. The query is embedded with `task="query"` so providers use their asymmetric query-side encoding. Dense catches the paraphrase BM25 is blind to.

Crucially, dense *degrades gracefully*. If the partition for that space has not been built, or a hosted embedder has no API key, `retrieve` catches the `FileNotFoundError` / `RuntimeError` and falls back to BM25-only rather than crashing the whole retriever. (A genuine bug like a bad model name still raises `ValueError` — we deliberately do *not* swallow that.) So the free/offline path always returns *something*.

6.3 Why both, on our corpus

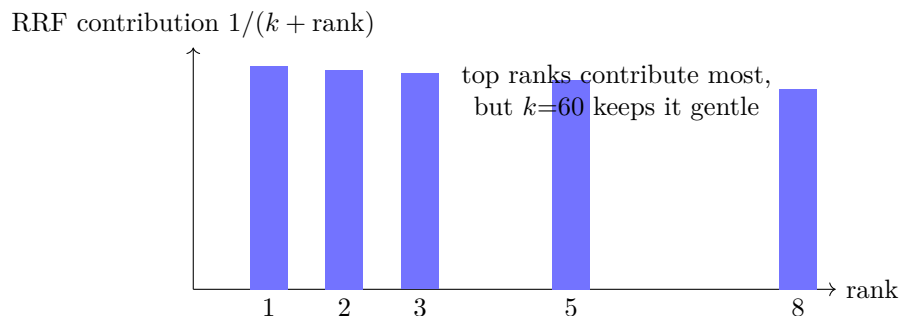
The notebook entries are terse and idiosyncratic. Sometimes the only handle is an exact name and BM25 wins; sometimes it is a concept the writer phrased unusually and dense wins. Running both and fusing them is strictly safer than betting on either alone — which is the entire motivation for the next section.

7 RRF: fusing ranked lists without calibrating their scores

Here is the trap. BM25 scores might run 0–15; cosine similarities run -1 – 1 . If we simply *add* them, whichever retriever happens to emit bigger numbers silently dominates the fusion. **Reciprocal Rank Fusion** sidesteps this entirely: it throws the raw scores away and uses only each item's *rank position*.

$$\text{RRF}(d) = \sum_{\ell \in \text{lists}} \frac{w_{\ell}}{k + \text{rank}_{\ell}(d)}.$$

A passage near the top of *either* list earns a big contribution; the constant k (default 60, the published default) damps the very top ranks so no single retriever can unilaterally decide the winner. The figure shows the shape of $1/(k + \text{rank})$: front-loaded, but never explosive.



RRF is the project’s universal combiner. It fuses BM25 with dense (here), it fuses *across queries* in RAG-Fusion (§9), and it is exactly how a future ColPali image-retrieval list would be merged with the text retrievers (§12.2) — one clean, scale-free operation everywhere.

8 Rerank: the expensive judge that reads query and passage together

First-stage retrieval is cheap and casts a wide net (“give me 50 candidates”). A *cross-encoder reranker* then re-reads the query *jointly* with each candidate and emits a single, far sharper relevance score — the difference between “these words co-occur” and “this passage actually answers the question.” Because that joint read is expensive, we feed it only `rerank_pool` (default 30) candidates: *retrieve fifty, rerank to a precise few*.

The reranker is the third swappable model axis (`lib/providers/rerank.py`), alongside embedding and generation:

- **local bge-reranker** via `fastembed` — free, offline, runs now;
- **Voyage rerank-2.5 / Cohere rerank-4-pro** — real adapters, but *paid*: key-gated, and every call is cost-logged.

After reranking, the reranker’s score becomes the `Hit`’s authority and we re-sort; any candidates beyond the pool keep their fused order behind the reranked ones.

9 Query transforms: HyDE and RAG-Fusion

Both transforms are *paid* LLM calls, both default off, both cost-logged inside `llm.generate`.

9.1 HyDE — embed a hypothetical *answer*

A question and its answer are written in different words. “Who was cured?” shares almost no vocabulary with “Mrs. Panyard’s cancer was reported cured after enrollment in the Seraphic Mass Association.” **HyDE** (Hypothetical Document Embeddings) asks the LLM to *write* a short, confident, in-style answer passage and then embeds *that* instead of the question. The fake answer lives in answer-space, nearer the real passages, so dense recall improves. If the LLM returns nothing, we fall back to embedding the raw query — the transform can only help, never break.

9.2 RAG-Fusion — many phrasings, one fused result

Different phrasings surface different true passages: one reader asks about a *favor*, another a *miracle*, another an *answered prayer*. `multi_queries` asks the LLM for a few diverse paraphrases (always keeping the original), we retrieve for each, and RRF-fuses the lists — the same combiner from before, now *across queries*. JSON parsing is forgiving: a malformed reply simply yields “no extra variants,” never a crash.

10 Self-RAG / CRAG: earning the right to answer

In an archive, a confident-but-unsupported answer is the worst possible outcome — it launders a guess into something that looks like a citation. So after retrieval we *grade* the result set before the generator ever sees it.

10.1 The free grader (`grade_context`)

It uses only signals we already have, so it costs nothing:

- **Top fused score** and the **margin** between #1 and #3 — a flat distribution means the retrieval is ambiguous, with no clear winner.
- **Lexical overlap** between the query terms and the top hits — did we even match the words we were asked about?
- **OCR min_conf** on the cited regions — evidence transcribed at low confidence is shaky no matter how well it ranks. This signal is unique to a *scanned* archive and we get it for free from the region geometry.

It returns a CRAG-style verdict — **answer** (strong match, adequate overlap), **caveat** (present but shaky: low OCR confidence or flat ranking), or **abstain** (top score or overlap too low) — plus a blended *confidence* number. The thresholds are deliberately conservative: for an archive, prefer “not found” to a fabrication.

Read the confidence number for what it is

The **confidence** returned by `grade_context` is a heuristic blend of overlap, rank margin, and OCR confidence — *not* a probability and *not* a correctness guarantee. It is a guardrail for *when to abstain*, to be tuned against a golden Q&A set (the evaluation work in the research plan, Part E).

10.2 The gated per-document grader (`grade_document`)

This is the Self-RAG “ISREL” idea: hand the LLM one (query, passage) pair and get back a relevance label, used to *filter* a reranked set before generation. It is paid and off by default. If the judge call fails, it *keeps* the document (labelled **partial**) and lets a human decide — a grader must never silently drop evidence.

11 Putting it together: `route_and_retrieve`

The adaptive entry point runs the brain the way the agent would:

1. **Route** (free heuristic unless `use_llm_router`); the route may *suggest* multi-query, honoured only if the caller enabled it or set `auto_apply_route`.
2. **Transform**: HyDE *or* multi-query, if enabled/escalated.
3. **Retrieve** per query via `retrieve` (itself BM25 + dense + RRF + optional rerank), then RRF-fuse *across* queries (RAG-Fusion), optionally reranking the fusion.
4. **Grade** the final set so the generator knows whether to answer, caveat, or abstain.

It returns a complete, serialisable result — `{query, route, queries_used, hits, grade}` — and with default config the *entire function makes zero paid calls*. Each model touchpoint is individually gated and logged.


```

from lib.retrieval import RetrievalConfig, route_and_retrieve

res = route_and_retrieve("What favors were reported for cancer?") # FREE
print(res["grade"]["verdict"], res["grade"]["confidence"]) # answer/caveat/abstain
for h in res["hits"]:
    p = h["provenance"]
    print(h["score"], h["kind"], p["doc_id"], "rid=", p["rid"], "pg=", p["page"])

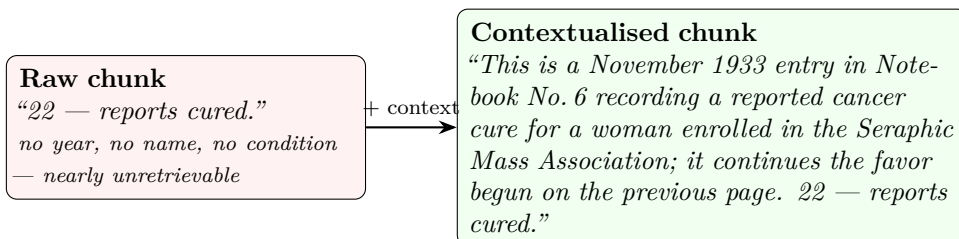
```

12 The frontier: three techniques this corpus wants next

The following are documented here and ranked by leverage in RESEARCH_PLAN.md (Part D), but are *not* yet coded in `retrieval.py` — they are either upstream (indexing-time) or heavy (a GPU model / an offline LLM build), and are intentionally deferred with clear TODOs. This section is explicit about wired-versus-planned so nobody assumes a toggle exists that does not.

12.1 Contextual Retrieval — the biggest win, sitting *upstream* of this file

Anthropic’s Contextual Retrieval reports a **−67% reduction in retrieval failures** from one move: prepend a 2–3 sentence, LLM-generated context to *each chunk* before embedding it *and* before BM25 indexing. Our notebooks are precisely the case it fixes. Recall the real entry:



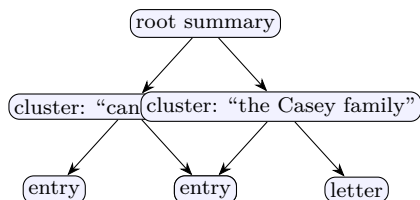
Now both BM25 (the literal words *November, 1933, cancer, Seraphic*) and the embedding (the meaning) can find it. Because the context is baked into the *stored* chunk, this is an **indexing-time** change: it belongs in the chunk/embed stage, and `retrieval.py` then benefits for free with no code change. It also dovetails with the structural work in RESEARCH_PLAN.md Part A (cross-page continuation, date reconstruction): those stages *recover* exactly the date / notebook / continuation context this technique injects. **This is the recommended next build.**

12.2 Multimodal / ColPali — retrieve over the page *images*

ColPali / ColQwen embed a page *image* as a grid of visual patch vectors and score relevance with *late interaction* (MaxSim): for each query token, take its best-matching image patch, then sum those bests. No OCR is involved. On scanned documents, visual recall routinely beats text (roughly 84% vs 62% in published comparisons) because it sees marginalia, layout, and precisely the strokes OCR drops. The plan: run ColPali as its *own* retrieval tool over our masked page PNGs, two-phase (a mean-pooled coarse screen, then MaxSim rerank of the top ~ 100), and **RRF-fuse** its ranked list with the text retrievers — the same combiner from before. *Why for us*: every chunk already carries `pdf_page` provenance, so an image hit cites the same page; and a handwritten archive is exactly where OCR misses hurt most. The heavy GPU model is deferred with a TODO — nothing here downloads or runs it.

12.3 RAPTOR — a tree of summaries for global questions

Single-passage retrieval answers “what happened to Mrs. Panyard?” but flounders on “what *conditions* recur across the notebooks?” — no one chunk holds that answer. **RAPTOR** recursively *clusters and summarises* the corpus into a tree: leaves are chunks, parents summarise clusters, the root summarises the summaries. Retrieval then works *coarse-to-fine* — a thematic query can match a high-level summary node and drill down — a cheaper path to global questions than full GraphRAG (the literature reports $\sim +20\%$ on long-document QA). The summaries are LLM-built in a one-time, cost-logged offline pass, so this is a *new index plus a new retriever node* that the router would prefer for complex aggregate asks. Deferred with a TODO.



13 How to run it, and the honest caveats

Recommended settings

- **Free smoke test:** `python -m lib.retrieval "your question"` — prints the route, the grade, the hits with citations, and a \$0 cost ledger.
- **Pre-filter for free** with `where`, e.g. `where={"kind": "notebook_entry"}` or a callable year-predicate over the date string, to narrow the search before ranking.
- **Dial up quality** by opting into spend: `use_rerank` (local free, cloud paid), `use_multiquery` (RAG-Fusion), `auto_apply_route` (let the agent escalate).

Caveats — read before trusting a result or a toggle

- **Dense needs an index.** `use_dense=True` does nothing until that space’s vector partition exists; `retrieve` quietly degrades to BM25-only by design — you see lexical results, not an error.
- **The local reranker is not the named one.** `fastembed` packages `bge-reranker-base`, not the `bge-reranker-v2-m3` named in config; the local path substitutes the base model. Wire FlagEmbedding / sentence-transformers for the exact weights (the heavy torch dependency is deliberately not auto-installed).
- **Grading is a guardrail, not ground truth.** `grade_context` returns a heuristic confidence, not a probability; tune the thresholds against a golden Q&A set.
- **Contextual / ColPali / RAPTOR are documented, not yet coded here.** They are upstream or heavy and are deferred with TODOs — this packet is explicit about wired-versus-planned.
- **Prices in `config.py` are placeholders.** Nothing is billed until a stage runs, and David has not said go. Confirm provider pricing before any billed sweep.

14 Sources

- Contextual Retrieval (−67% failures): <https://www.datacamp.com/tutorial/contextual-retrieval-anthr>
<https://www.freecodecamp.org/news/how-contextual-embeddings-and-hybrid-search-fix-retrieval>
- Advanced RAG, hybrid + RRF + rerank: <https://atlan.com/know/advanced-rag-techniques/>;
<https://machinelearningmastery.com/beyond-vector-search-5-next-gen-rag-retrieval-strategies>
- RAPTOR: <https://ragflow.io/blog/long-context-rag-raptor>; RAG vs GraphRAG eval:
<https://arxiv.org/html/2502.11371v3>
- Multimodal / ColPali (late interaction): <https://weaviate.io/blog/late-interaction-overview>;
https://huggingface.co/learn/cookbook/multimodal_rag_using_document_retrieval_and_reranker_and_vlms
- Adaptive / router RAG (−28% cost, +8% accuracy): <https://arxiv.org/pdf/2604.03455>;
<https://www.llamaindex.ai/blog/rag-is-dead-long-live-agentic-retrieval>
- Self-RAG / CRAG (grade, reflect, abstain), RAG 2026 blueprint: https://dev.to/suraj_khaitan_f893c243958/-rag-in-2026-a-practical-blueprint-for-retrieval-augmented-generation-1

See RESEARCH_PLAN.md Part D for the full ranked frontier and the tiered roadmap (Tier 0 = this module: contextual retrieval + hybrid + RRF + rerank + adaptive router + cited generation + evaluation).